

Implementation Blueprint — Days 2 to 10

Project: PrepGenie (AI Resume Analyzer & Mock Interview Coach)

DAY 2 — Project Scaffolding, Auth, DB Schema

Objective

Set up the full project skeleton, connect Supabase, implement authentication, and design the database schema.

Learning Goals

- Monorepo/folder structuring for full-stack apps
- Supabase Auth (OAuth + email/password)
- Environment variable management & secrets hygiene
- Postgres schema design

Features Delivered

- Working signup/login/logout flow
- Protected route wrapper
- Database tables created

Step-by-Step Plan

1. Create GitHub repo (`prepgenie`) with (`.gitignore`), MIT (`LICENSE`).
2. Scaffold frontend: (`npm create vite@latest client -- --template react`)
3. Scaffold backend: (`mkdir server && npm init -y`) → install (`express`), (`cors`), (`dotenv`), (`@supabase/supabase-js`)
4. Create Supabase project → enable Email + Google Auth providers.
5. Design & run SQL for tables: (`users`), (`resumes`), (`interview_sessions`), (`progress_snapshots`) (see PRD §8).
6. Implement Supabase client in (`client/src/lib/supabaseClient.js`).
7. Build (`Login.jsx`), (`Signup.jsx`) pages with Tailwind forms.
8. Implement (`AuthContext.jsx`) using React Context + (`onAuthStateChange`).
9. Build (`ProtectedRoute.jsx`) wrapper component.
10. Set up React Router with routes: (`/`), (`/login`), (`/signup`), (`/dashboard`), (`/analyzer`), (`/interview`).

Files/Folders to Create

```
prepgenie/  
├─ client/  
│   └─ src/  
│       └─ lib/supabaseClient.js  
│       └─ context/AuthContext.jsx  
│       └─ components/ProtectedRoute.jsx  
│       └─ pages/Login.jsx  
│       └─ pages/Signup.jsx  
│       └─ pages/Landing.jsx  
│       └─ App.jsx  
├─ server/  
│   └─ index.js  
│   └─ routes/  
│   └─ .env  
├─ README.md  
└─ .gitignore
```

APIs / Libraries / Tools

- Supabase (Auth + Postgres)
- React Router v6
- Tailwind CSS + shadcn/ui setup
- dotenv, cors, express

Testing Tasks

- Sign up a test user, confirm row appears in Supabase `auth.users`
- Try accessing `/dashboard` while logged out → should redirect to `/login`
- Refresh page while logged in → session should persist

Debugging Tips

- If Google OAuth redirect fails, double-check the Supabase redirect URL matches your local `http://localhost:5173`
- CORS errors from backend → confirm `cors()` middleware is applied before routes
- Env vars not loading → ensure `.env` is in root of `server/` and `dotenv.config()` runs at top of `index.js`

End-of-Day Checklist

- ☐ Repo initialized and pushed to GitHub
- ☐ Supabase project created with 4 tables
- ☐ Auth flow (signup/login/logout) working end-to-end
- ☐ Protected routes redirect correctly
- ☐ `.env` files excluded from git

Expected Screenshots

- Landing page
- Signup form
- Supabase table editor showing schema
- Successful login redirecting to empty dashboard shell

Handoff Notes for Day 3

Auth and DB are ready. Day 3 will build the resume upload + parsing pipeline. Ensure `resumes` table and Supabase Storage bucket (`resumes-bucket`) are ready to receive files.

DAY 3 — Resume Upload & Parsing Pipeline

Objective

Allow users to upload a resume file and extract clean text server-side.

Learning Goals

- File upload handling (multipart/form-data)
- PDF/DOCX text extraction
- Supabase Storage buckets

Features Delivered

- Resume upload UI with drag-and-drop
- Backend endpoint that extracts and returns raw text
- File stored in Supabase Storage; metadata stored in `resumes` table

Step-by-Step Plan

1. Create Supabase Storage bucket `resumes-bucket` (private).
2. Install backend libs: `multer`, `pdf-parse`, `mammoth`.
3. Build `POST /api/resume/upload` endpoint:
 - Accept file via `multer`
 - Detect file type (.pdf vs .docx)
 - Extract text using `pdf-parse` or `mammoth`
 - Upload original file to Supabase Storage
 - Insert row into `resumes` table with `raw_text`
 - Return `resume_id` + extracted text preview to frontend
4. Build frontend `Analyzer.jsx` page with drag-and-drop upload zone (use `react-dropzone`).
5. Show upload progress state and extracted text preview after upload.
6. Add file validation: max 5MB, only `.pdf`/`.docx`.

Files/Folders to Create/Modify

```
server/routes/resumeRoutes.js
server/utils/parseResume.js
client/src/pages/Analyzer.jsx
client/src/components/UploadDropzone.jsx
```

APIs / Libraries / Tools

- multer (file upload middleware)
- pdf-parse, mammoth (text extraction)
- react-dropzone
- Supabase Storage SDK

Testing Tasks

- Upload a sample PDF resume → confirm text extracted correctly in console/logs
- Upload a DOCX resume → confirm mammoth extraction works
- Try uploading a >5MB file → should show validation error
- Try uploading a .jpg → should be rejected

Debugging Tips

- `pdf-parse` fails on scanned/image-only PDFs — catch this and show a friendly error message
- Multer "unexpected field" errors usually mean frontend field name doesn't match backend `upload.single('resume')` key
- Supabase Storage upload requires correct MIME type header

End-of-Day Checklist

- ☐ Upload endpoint working for both PDF and DOCX
- ☐ File stored in Supabase Storage
- ☐ `resumes` table row created with raw text
- ☐ Frontend shows extracted text preview
- ☐ Validation errors handled gracefully

Expected Screenshots

- Drag-and-drop upload UI
- Upload success state with text preview
- Supabase Storage bucket showing uploaded file
- Error state for invalid file type

Handoff Notes for Day 4

Raw resume text is now available in DB. Day 4 will send this text to the Claude API for structured analysis and render the results as a UI report.

DAY 4 — AI Resume Analysis Engine + UI

Objective

Send extracted resume text to the AI API and render a structured, visual feedback report.

Learning Goals

- Prompt engineering for structured JSON output
- Backend AI API integration (keeping keys server-side)
- Rendering dynamic AI output as polished UI components

Features Delivered

- `/api/resume/analyze` endpoint calling Claude API
- ATS score badge, keyword gap list, section-wise suggestions UI
- Feedback stored in DB (`feedback_json`, `ats_score` columns)

Step-by-Step Plan

1. Write a strict system prompt (see below) enforcing JSON-only output.
2. Build `server/services/aiService.js` with a function `analyzeResume(resumeText, targetRole)`.
3. Call Anthropic API (`claude-sonnet-5`) with `max_tokens` ~1500, temperature 0.3.
4. Parse and validate JSON response (use a try/catch + schema check; retry once on parse failure).
5. Update `resumes` row with `ats_score` and `feedback_json`.
6. Build `AnalysisReport.jsx` component:
 - Circular ATS score gauge (0-100)
 - Missing keywords as chips/tags
 - Section-wise accordion (Summary, Skills, Projects, Experience) with suggestions
 - "Re-analyze" button
7. Add loading skeleton while AI call is in progress (8-10s expected).

Example System Prompt (for aiService.js)

You are an expert technical resume reviewer and ATS system.
Given a resume's raw text and a target job role, return ONLY valid JSON
(no markdown, no prose) with this exact shape:

```
{
  "ats_score": number (0-100),
  "missing_keywords": string[],
  "sections": {
    "summary": { "score": number, "suggestions": string[] },
    "skills": { "score": number, "suggestions": string[] },
    "projects": { "score": number, "suggestions": string[] },
    "experience": { "score": number, "suggestions": string[] }
  },
  "overall_feedback": string
}
```

Files/Folders to Create/Modify

```
server/services/aiService.js
server/routes/resumeRoutes.js (add /analyze route)
client/src/components/AnalysisReport.jsx
client/src/components/ScoreGauge.jsx
client/src/components/SkeletonLoader.jsx
```

APIs / Libraries / Tools

- Anthropic Claude API (server-side only)
- recharts or a custom SVG gauge for score visualization
- shadcn/ui Accordion component

Testing Tasks

- Analyze 3 different sample resumes (strong, average, weak) → confirm reasonable score differentiation
- Force a malformed AI response (e.g., truncate max_tokens) → confirm retry/error handling works
- Confirm feedback persists in DB and reloads on page refresh

Debugging Tips

- If AI adds markdown fences (``json) around JSON, strip them with a regex before `JSON.parse`
- Keep `max_tokens` generous (1200-1500) to avoid truncated JSON
- Log raw AI responses server-side during development to debug prompt issues

End-of-Day Checklist

- ☐ Analyze endpoint returns valid structured JSON consistently
- ☐ ATS score gauge renders correctly
- ☐ Section suggestions render in accordion UI
- ☐ Feedback persisted and reloadable
- ☐ Loading and error states implemented

Expected Screenshots

- Analysis report page with score gauge + keyword chips
- Section accordion expanded showing suggestions
- Loading skeleton mid-analysis

Handoff Notes for Day 5

Resume analyzer module is feature-complete. Day 5 shifts to building the mock interview question-generation engine, reusing `aiService.js` patterns established today.

DAY 5 — Mock Interview: Question Generation Engine

Objective

Build the backend logic that generates role-specific, resume-aware interview questions.

Learning Goals

- Multi-turn conversational AI design
- Role-based prompt templating
- Session state management in Postgres

Features Delivered

- Role selection screen (SDE Intern / Data Analyst / Core Engineering / HR Round)
- `/api/interview/start` endpoint generating 5-8 questions
- Interview session created and stored in DB

Step-by-Step Plan

1. Build `RoleSelect.jsx` UI — card-based role picker with icons/descriptions.
2. Build `server/services/interviewService.js` with `generateQuestions(role, resumeText)`.
3. Craft prompt combining role + resume highlights → generate mixed question set (2 technical, 2 behavioral, 1-2 resume-specific, 1 HR-style).
4. Enforce JSON output: `{ "questions": [{ "id", "type", "question" }] }`
5. Create `interview_sessions` row with `role`, `questions_json`, empty `answers_json`.
6. Return `session_id` + first question to frontend.
7. Build `InterviewChat.jsx` shell UI (chat bubble layout, question shown one at a time).

Files/Folders to Create/Modify

```
server/services/interviewService.js
server/routes/interviewRoutes.js
client/src/pages/RoleSelect.jsx
client/src/pages/InterviewChat.jsx
client/src/components/ChatBubble.jsx
```

APIs / Libraries / Tools

- Claude API (temperature 0.6–0.7 for variety)
- Framer Motion for chat bubble entrance animation

Testing Tasks

- Generate questions for all 4 roles → confirm variety and relevance
- Regenerate for same role/resume → confirm reasonable variation (not identical every time)
- Confirm session row correctly stores role + questions

Debugging Tips

- If questions feel generic, inject 2–3 explicit resume keywords into the prompt to force personalization
- Keep question count fixed (e.g., exactly 6) by explicitly stating "Return exactly 6 questions" in the prompt
- Validate array length after parsing; regenerate if count is off

End-of-Day Checklist

- ☐ Role selection UI complete
- ☐ Question generation working for all roles
- ☐ Session persisted with questions
- ☐ Chat UI shell displays first question

Expected Screenshots

- Role selection cards
- Chat UI showing first AI question in a chat bubble

Handoff Notes for Day 6

Questions are generating correctly. Day 6 builds the full interactive chat flow (user answering, moving through all questions).

DAY 6 — Interview Chat UI & Answer Submission Flow

Objective

Build the complete interactive mock-interview experience where users answer each question sequentially.

Learning Goals

- Complex state management for multi-step flows
- Optimistic UI updates
- Handling sequential async operations gracefully

Features Delivered

- Full chat-style Q&A flow
- Text input (with optional Web Speech API voice-to-text)
- Progress indicator ("Question 3 of 6")
- Answers submitted and stored per question

Step-by-Step Plan

1. Build state machine in `InterviewChat.jsx`: `currentQuestionIndex`, `answers[]`, `isSubmitting`.
2. On answer submit → `POST /api/interview/answer` with `session_id`, `question_id`, `answer_text`.
3. Backend appends answer to `answers_json` in the session row.
4. Advance to next question automatically; show typing indicator between questions for realism.
5. Add optional voice input using the browser's `SpeechRecognition` API with a mic button.
6. Add "Skip question" and "End interview early" options.
7. On final question answered → call `/api/interview/complete` to trigger scoring (built Day 7) and redirect to results page (stub for now).

Files/Folders to Create/Modify

```
client/src/pages/InterviewChat.jsx (expand)
client/src/hooks/useSpeechToText.js
server/routes/interviewRoutes.js (add /answer, /complete)
```

APIs / Libraries / Tools

- Web Speech API (`SpeechRecognition`)
- React state/hooks (`useReducer`) recommended for this complex flow)

Testing Tasks

- Complete a full 6-question interview via text input
- Test voice-to-text on a supported browser (Chrome)
- Test "skip" and "end early" flows
- Confirm all answers persist correctly in DB after each submission

Debugging Tips

- `SpeechRecognition` is not supported in Firefox — feature-detect and hide the mic button if unsupported
- Use `useReducer` instead of multiple `useState` calls to avoid race conditions when advancing questions
- Debounce the submit button to prevent double-submission

End-of-Day Checklist

- ☐ Full Q&A flow works end-to-end for all 6 questions
- ☐ Answers persist correctly after each step
- ☐ Voice-to-text works as a stretch feature (or gracefully degrades)
- ☐ Skip/end-early flows work without breaking state

Expected Screenshots

- Mid-interview chat screen with progress indicator
- Voice input active state
- "Interview complete" transition screen

Handoff Notes for Day 7

All answers are now captured per session. Day 7 builds the AI scoring engine that evaluates these answers and generates the results report.

DAY 7 — AI Answer Scoring & Feedback Engine

Objective

Score the full interview session using AI and generate a results report.

Learning Goals

- Designing consistent AI scoring rubrics
- Aggregating multiple AI evaluations into a summary
- Building results/report UI with charts

Features Delivered

- `/api/interview/complete` scores all Q&A pairs
- Results page: overall score, per-category breakdown, weak topics, improvement tips

Step-by-Step Plan

1. Build `scoreInterview(questions, answers)` in `interviewService.js`.
2. Prompt AI to score EACH answer on: Correctness (0-10), Clarity (0-10), Confidence (0-10), plus a 1-line tip.
3. Enforce JSON output: `{ "per_question": [...], "overall_score": number, "weak_topics": string[], "summary": string }`.

4. Store `scores_json` in `interview_sessions` row; also insert a row into `progress_snapshots`.
5. Build `InterviewResults.jsx`:
 - Overall score badge
 - Bar chart of Correctness/Clarity/Confidence averages (recharts)
 - List of weak topics with tips
 - "Retry Interview" and "Back to Dashboard" buttons

Files/Folders to Create/Modify

```
server/services/interviewService.js (add scoreInterview)
server/routes/interviewRoutes.js (/complete route)
client/src/pages/InterviewResults.jsx
client/src/components/ScoreBarChart.jsx
```

APIs / Libraries / Tools

- Claude API (temperature 0.3 for consistent scoring)
- recharts (BarChart/RadarChart)

Testing Tasks

- Complete interviews with strong vs weak sample answers → confirm score differentiation makes sense
- Confirm `progress_snapshots` row is created for dashboard use later
- Test scoring with very short ("I don't know") answers → should score low, not crash

Debugging Tips

- If scores feel inconsistent across runs, lower temperature further (0.2) and add explicit scoring anchors in the prompt (e.g., "0-3 = no relevant content, 4-6 = partial, 7-10 = strong")
- Batch all Q&A pairs into a single AI call (not one call per question) to save cost/time and maintain context

End-of-Day Checklist

- ☐ Scoring engine returns consistent, structured results
- ☐ Results page renders scores and weak topics clearly
- ☐ `progress_snapshots` correctly populated
- ☐ Retry flow works (creates a new session)

Expected Screenshots

- Interview results page with score chart
- Weak topics list with tips

Handoff Notes for Day 8

Both core AI modules (resume + interview) are complete end-to-end. Day 8 builds the unified dashboard that ties both together with historical charts.

DAY 8 — Dashboard: Analytics, Charts, History

Objective

Build a unified dashboard showing the user's overall prep journey across both modules.

Learning Goals

- Data aggregation queries
- Multi-chart dashboard layout design
- Empty-state UX design

Features Delivered

- Resume score history line chart
- Interview performance radar chart (Correctness/Clarity/Confidence averages over time)
- Weak topics aggregated list
- Streak tracker widget
- Recent activity list (last 5 resumes/interviews)

Step-by-Step Plan

1. Build `GET /api/dashboard/summary` endpoint aggregating data from `resumes`, `interview_sessions`, `progress_snapshots`.
2. Build `Dashboard.jsx` layout with a responsive grid (shadcn/ui `Card` components).
3. Implement `ResumeHistoryChart.jsx` (line chart, recharts `LineChart`).
4. Implement `SkillRadarChart.jsx` (recharts `RadarChart` for Correctness/Clarity/Confidence).
5. Implement `StreakTracker.jsx` — simple calculated streak based on `created_at` dates with consecutive-day logic.
6. Design empty states for new users (no resume/interview yet) with clear CTAs.

Files/Folders to Create/Modify

```
server/routes/dashboardRoutes.js
client/src/pages/Dashboard.jsx
client/src/components/ResumeHistoryChart.jsx
client/src/components/SkillRadarChart.jsx
client/src/components/StreakTracker.jsx
```

APIs / Libraries / Tools

- recharts (LineChart, RadarChart)

- date-fns (streak date calculations)

Testing Tasks

- Test dashboard with a brand-new account (empty states must look intentional, not broken)
- Test with a seeded account (3+ resumes, 3+ interviews) → confirm charts render correctly
- Verify streak logic across multiple days (may need to manually adjust `created_at` in DB for testing)

Debugging Tips

- Recharts requires a fixed-height parent container (`<div style={{height: 300}}>`) or charts collapse to 0px
- For streak logic, normalize dates to UTC midnight before comparing to avoid timezone off-by-one bugs

End-of-Day Checklist

- ☐ Dashboard aggregation endpoint working
- ☐ Both charts rendering with real data
- ☐ Streak tracker calculates correctly
- ☐ Empty states designed for new users

Expected Screenshots

- Full dashboard view (populated)
- Empty-state dashboard view (new user)

Handoff Notes for Day 9

All features are functionally complete. Day 9 is dedicated to UI polish, responsiveness, and edge-case handling across the entire app.

DAY 9 — UI Polish, Error Handling, Edge Cases, Responsive Design

Objective

Elevate the app from "functional" to "portfolio-impressive" through design polish and robustness.

Learning Goals

- Production-grade error boundary patterns
- Responsive design auditing across breakpoints
- Micro-interactions and perceived-performance techniques

Features Delivered

- Consistent design system (colors, spacing, typography) across all pages
- Global error boundary + toast notification system

- Fully responsive layouts (mobile/tablet/desktop)
- Loading, empty, and error states standardized everywhere

Step-by-Step Plan

1. Audit and standardize a Tailwind design system: consistent spacing scale, font sizes, brand color palette, dark mode (stretch goal).
2. Add a global `ErrorBoundary.jsx` around the app.
3. Add `react-hot-toast` for success/error notifications across all AI calls.
4. Add subtle Framer Motion transitions between routes/pages.
5. Test every page at 375px (mobile), 768px (tablet), 1440px (desktop) — fix layout breaks.
6. Handle edge cases:
 - AI API timeout/rate limit → friendly retry message
 - Empty resume upload → validation
 - Network offline → offline banner
7. Add a polished 404 page and a global navbar/footer.
8. Run a full accessibility pass (labels, alt text, contrast, keyboard nav).

Files/Folders to Create/Modify

```
client/src/components/ErrorBoundary.jsx
client/src/components/Navbar.jsx
client/src/components/Footer.jsx
client/src/pages/NotFound.jsx
client/src/styles/theme.css (or tailwind.config.js tokens)
```

APIs / Libraries / Tools

- react-hot-toast
- Framer Motion
- Chrome DevTools device toolbar + Lighthouse audit

Testing Tasks

- Run Lighthouse audit (target: 90+ performance, 90+ accessibility)
- Manually test all flows on mobile viewport
- Simulate slow 3G network in DevTools to verify loading states hold up
- Simulate AI API failure (wrong API key temporarily) → confirm graceful error toast, no white-screen crash

Debugging Tips

- White-screen crashes are almost always an uncaught error in a child component — wrap risky components in the ErrorBoundary

- If mobile layout breaks, check for fixed pixel widths instead of `%/max-w-*` Tailwind classes
- Use Lighthouse's specific suggestions rather than guessing at performance fixes

End-of-Day Checklist

- ☐ Lighthouse scores 90+ on performance & accessibility
- ☐ All pages responsive at 3 breakpoints
- ☐ Global error handling in place, no unhandled crashes
- ☐ Toast notifications integrated for all async actions
- ☐ 404 page and consistent navbar/footer present

Expected Screenshots

- Mobile view of dashboard and interview chat
- Toast notification examples (success + error)
- Lighthouse audit score screenshot

Handoff Notes for Day 10

App is feature-complete and polished. Day 10 is dedicated purely to deployment, documentation, and packaging for recruiters.

DAY 10 — Testing, Deployment, README, Demo Packaging

Objective

Ship the final product publicly and package it for maximum recruiter/GitHub impact.

Learning Goals

- Production deployment workflows (Vercel + Render)
- Writing a recruiter-friendly README
- Creating a compelling project demo

Features Delivered

- Live public deployment
- Polished GitHub repository
- Demo video/GIFs and screenshots
- LinkedIn-ready project post draft

Step-by-Step Plan

1. **Final QA pass:** run through the entire user journey (signup → resume upload → analysis → mock interview → results → dashboard) exactly as a recruiter would.
2. Deploy backend to Render (or Railway): set environment variables (`SUPABASE_URL`, `SUPABASE_KEY`, `ANTHROPIC_API_KEY`, `CORS_ORIGIN`).
3. Deploy frontend to Vercel: set `VITE_API_URL` to the deployed backend URL, `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`.

4. Update Supabase Auth redirect URLs to include the production domain.
5. Smoke-test the deployed app end-to-end (not just localhost).
6. Write the final `README.md`:
 - Project title + one-line pitch
 - Live demo link + screenshots/GIF
 - Problem statement
 - Features list
 - Tech stack badges
 - Architecture diagram (simple draw.io or Excalidraw export)
 - Local setup instructions
 - Future roadmap
7. Record a 60-90 second demo screen recording (Loom or OBS) showing the core flow.
8. Draft a LinkedIn post: problem → solution → tech stack → link → tag #buildinpublic #AI #placements.
9. Tag a GitHub release `v1.0.0`.

Files/Folders to Create/Modify

```
README.md (final version)
docs/architecture-diagram.png
docs/demo.gif
.env.example (both client and server)
```

APIs / Libraries / Tools

- Vercel CLI/dashboard
- Render/Railway dashboard
- Excalidraw or draw.io (architecture diagram)
- Loom / OBS Studio (demo recording)

Testing Tasks

- Full smoke test on the live production URL (not localhost) across the entire user journey
- Test on a different device/network to confirm it's genuinely public and working
- Verify no secrets are committed to GitHub (`git log -p | grep -i "sk-"` style check, or use `gitleaks`)
- Confirm README renders correctly on GitHub (preview before final push)

Debugging Tips

- CORS errors in production usually mean the backend's allowed origin doesn't match the exact Vercel URL (including `https://`, no trailing slash)

- If Supabase Auth redirect fails in production, double check the exact production URL is whitelisted in Supabase Auth settings
- Environment variables not picking up on Vercel/Render require a redeploy after being set, not just a save

End-of-Day Checklist

- ☐ App fully deployed and publicly accessible
- ☐ End-to-end smoke test passed on production
- ☐ README complete with screenshots/GIF and architecture diagram
- ☐ No secrets committed to git history
- ☐ GitHub release tagged v1.0.0
- ☐ LinkedIn post drafted and ready to publish

Expected Screenshots

- Final deployed landing page (production URL visible in browser bar)
- README as rendered on GitHub
- Demo GIF thumbnail

Handoff Notes (Project Complete)

PrepGenie is now fully built, deployed, and documented. Recommended next steps beyond Day 10: gather 5-10 real users (classmates) for feedback, and consider the "Future Scope" items in the Pitch Deck document for a v2 iteration.